

VCF/BCF 多样本 .gsc 二进制结构

对应 Excel：VCF层次结构 / VCF_gsc二进制结构。重点看：文件开头的两个位置值如何找到 main archive -lossless part2 与 SDSL tail。

● 必选 ○ 条件/兼容 - - - -> 引用/跳转
黑色填充模块表示解压时必须先读取的入口。

文件字节布局

真实字节顺序；多样本 .gsc 没有顶层魔数/版本。

1. TOP_HEADER

固定 17 bytes：模式与两个位置值。
`bool + uint64_t + uint64_t`
入口字段本身不带魔数/版本。

2. MAIN_ARCHIVE

offset=17 到 other_fields_offset。
主要数据区，保存 GT 全局索引、元信息和 fixed chunks。

chunk_streams / basic params

copy 来源位流 / 范围索引

列分块 / 样本重排 / 元信息

fixed-fields chunk 内容

3. LOSSLESS_PART2

仅 lossless 文件存在。
[other_fields_offset, sdsi_offset]

part2_params + key_i_size/data

4. SDSL_TAIL

从 sdsi_offset 开始的 GT zero/copy 位图。
`zeros[2] + copies[2]`
每项均为 `sdsi::rrr_vector<>`

结构展开：按文件位置读取的 .gsc

中间留白只放跨面板箭头；右侧按 Excel 字段明细展开。

TOP_HEADER：文件入口字段

解压端先读取 3 个字段，然后找到 main archive -part2 和 SDSL tail 三个区域。

mode_type

other_fields_offset

sdsi_offset

lossless/lossy 解压模式必须匹配 mode_type。

MAIN_ARCHIVE：位置表 + GT 参数 + 元信息 + fixed chunks

这里按写入顺序串联；读取指定 fixed chunk 时，需要先查 chunk_streams 里的位置。

chunk_streams

chunks_streams_size
[actual_pos, offset] × N

basic + copy 来源

ploidy, row/col limits, vec_len
used_bits_cp + bitstream

范围信息 + 列分块

chunks_min_pos[] / where_chrom[]
n_col_blocks + column ranges

样本重排数据流

legacy: row_block_id + vint bytes
tiled: row_block_id + col_block_id + bytes

元信息

GSC_META_MAGIC + 压缩后端 + 压缩后的 header/sample names

fixed-fields chunk 内容

按 chunk 顺序拼接；最终 fixed chunk 无长度前缀；chunk 内部见 GSCF V2。

GSCF V2：Fixed Fields Chunk 内部

chunk_header

magic = 0x46435347 ("GSCF")
version = 2
total_variants
row_block_count, flags

row_block_directory

每个 row_block 一个目录项：
variant_count / first_pos / last_pos
chrom_off / chrom_size
pos_off / pos_size
idref/alt/qual off+size
gt_off / gt_size (V2)
范围查询直接按 first_pos/last_pos 选 row_block。

data_region (数据区)

目录中的 off/size 会指向这里的各字段内容。

内容顺序

CHROM -> POS -> ID -> REF -> ALT -> QUAL -> GT

字符串列：append_str(), NUL 结尾。

POS：append(int64 delta)，十进制 ASCII + NUL。

GT：稀疏索引内容 + 1-byte codec marker。

兼容分支：旧 fixed-fields chunk 不以 GSCF magic 起始，而是 [no_variants] + 7 个 [size,data]；无 row_block 目录。

LOSSLESS_PART2：FILTER / INFO / FORMAT 多流容器

part2_params

actual_variants, key table
压缩后端 + FORMAT 字典数据

key_i_size / key_i_data

每个 key 的 size/data 子流
FILTER / INFO / FORMAT

File_Handle_2 footer

stream_name, part 位置/大小, footer_size

SDSL_TAIL：GT 全零/copy 位图

zeros_only[0/1]

copies[0/1]

copy 来源差值位流位于 MAIN_ARCHIVE，不在 SDSL tail。

单样本 native gVCF 二进制结构

对应 Excel：gVCF层次结构 / gVCF_native二进制结构。重点看：显式文件头、重复 block 长度标记、文件尾 footer_offset 如何找到索引。

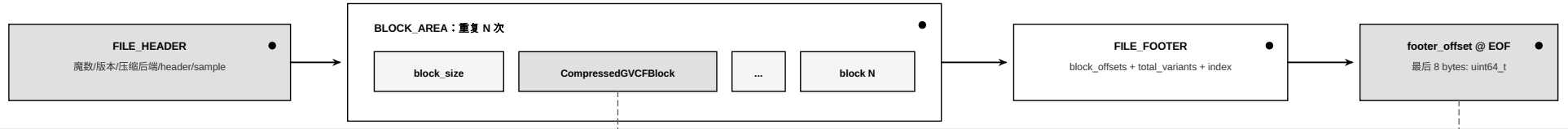
● 必选 ○ 版本/条件 ----> 引用/跳转
横向轴表示文件从起点到 EOF 的顺序。

文件顺序：Header -> Blocks -> Footer -> EOF 指针

native gVCF 与多样本 .gsc 不同：它明确写了 GVCF 魔数和版本，并通过文件最后 8 字节找到 footer。

文件整体布局

从左到右就是磁盘写入顺序；BLOCK_AREA 中每个 block 都自带长度前缀。



FILE_HEADER 展开

读取时先检查 GVCF_FILE_MAGIC，再按 version 读取后续字段。

固定入口

magic = 0x47564346 ("GVCF"), version = 4, backend_id, num_samples

header text

V3+：header_flag 决定 header 是压缩保存还是原样保存

sample_names + placeholder

重复 [name_size][name]；最后补一个 uint64 占位

CompressedGVCFBlock 内部结构

每个 block 内容里会再次写 block 魔数/版本，各字段都按 CompressedField 的格式保存。

block header

4-byte block magic
block_version
variant_count
sample_count, flags

known fields

chrom, pos, id, ref, alt
qual, filter, info_end
gt_*, dp, gq, min_dp
pl, ad

unknown fields

unknown_info_count
unknown_info items
unknown_format_count
unknown_format items

字段统一外层格式

CompressedField = method + original_count + data_size + data

data 内部可再带 backend_flag（压缩标记），用于区分原始字段数据与压缩后字段数据。

注意：unknown_info / unknown_format 使用 unordered_map 迭代，字段写出顺序不保证按名称排序。

FILE_FOOTER 与文件尾定位

解压或查询时先从 EOF 读取 footer_offset，再跳到 footer。

footer fixed area

num_blocks + block_offsets[] + total_variants
block_offsets 指向各 block_size 字段

block index (V4)

num_indices
[chrom_len][chrom][start_pos][end_pos]
[file_offset][variant_count] × N

footer_offset @ EOF

文件最后 8 字节保存 footer 起始偏移
范围查询先从这里找到索引区。

CompressedField 外层格式与典型内容

Excel 明细里的 CompressedField 外层格式和 FIELD_PAYLOADS_AND_METHODS 对应这里。

CompressedField 外层格式

1. method : uint8 FieldCompressionMethod
2. original_count : VarUInt
3. data_size : VarUInt
4. data : bytes

data 内部压缩标记

第 1 字节是 backend_flag（压缩标记）：
0 = 后面直接跟字段编码内容
1 = 后面跟压缩后的字段内容
空字段 data_size=0 时没有 backend_flag。

典型字段内容例子

POS: Delta [original_len][first_value][delta_count][delta...]
REF: exception list + packed 2-bit bases ; ALT: Mask(first ALT) + extra_alts 列表
PL/AD: v2 优化数据；GQ: PL 预测例外值
RLE / Delta / Mask / Dictionary 等基础编码由 gvcf_encoding.* 实现。